

3-Tier Web Application Project Documentation

This project demonstrates a complete 3-Tier Web Application architecture suitable for DevOps, Cloud, AWS, and GitHub portfolio projects. The application is divided into Presentation Layer, Application Layer, and Database Layer.

Project Overview

A scalable and secure web application following industry-standard 3-tier architecture. Designed for deployment using Docker, CI/CD pipelines, AWS services, and monitoring tools.

Architecture

Tier 1 – Presentation Layer (Frontend): HTML, CSS, JavaScript, React.

Tier 2 – Application Layer (Backend): Python Flask / Node.js REST APIs.

Tier 3 – Database Layer: MySQL/PostgreSQL database.

Project Features

- User Registration and Login
- CRUD Operations
- REST API Integration
- Docker Containerization
- CI/CD Pipeline using Jenkins/GitHub Actions
- AWS Deployment Support
- Monitoring and Logging

Technology Stack

Frontend: HTML, CSS, JavaScript, React

Backend: Flask / Node.js

Database: MySQL / PostgreSQL

Version Control: Git & GitHub

CI/CD: Jenkins, GitHub Actions

Cloud: AWS EC2, S3, RDS

Containerization: Docker

Folder Structure

```
project-root/  
├── frontend/  
├── backend/  
├── database/  
├── docker/  
├── docs/  
├── README.md  
└── Jenkinsfile
```

Deployment Workflow

1. Developer pushes code to GitHub.
2. CI/CD pipeline triggers automatically.
3. Application is built and tested.
4. Docker image is created.
5. Image is deployed to AWS EC2.
6. Database hosted on AWS RDS.
7. Monitoring enabled using CloudWatch.

GitHub README Content

Include project overview, architecture diagram, setup instructions, deployment steps, screenshots, and future enhancements.

Future Enhancements

- Kubernetes Deployment
- Terraform Infrastructure Automation
- DevSecOps Integration
- Auto Scaling and Load Balancing